Servlet Komponenten:
Servlet, Filter, Listener

Themen heute

- Komponente Servlet
  - Bemerkungen zum AB1.3
- **Servlet Context** und **Servlet Container**
  - Einführung
- Komponente **Filter**
  - Arbeitsblatt zum Filter
- Komponente **Listener**
  - Arbeitsblatt zu Listener und ServletContext

Besprechung Arbeitsblatt 1.3

- Request auf
  http://localhost:8080/flashcard-basic/app/questionnaires/1

- Lösungsschritte
  1. Dispatching des Requests
  2. Questionnaire ID aus dem Request extrahieren
  3. Questionnaire mit ID aus Repository lesen
  4. HTML-Response generieren

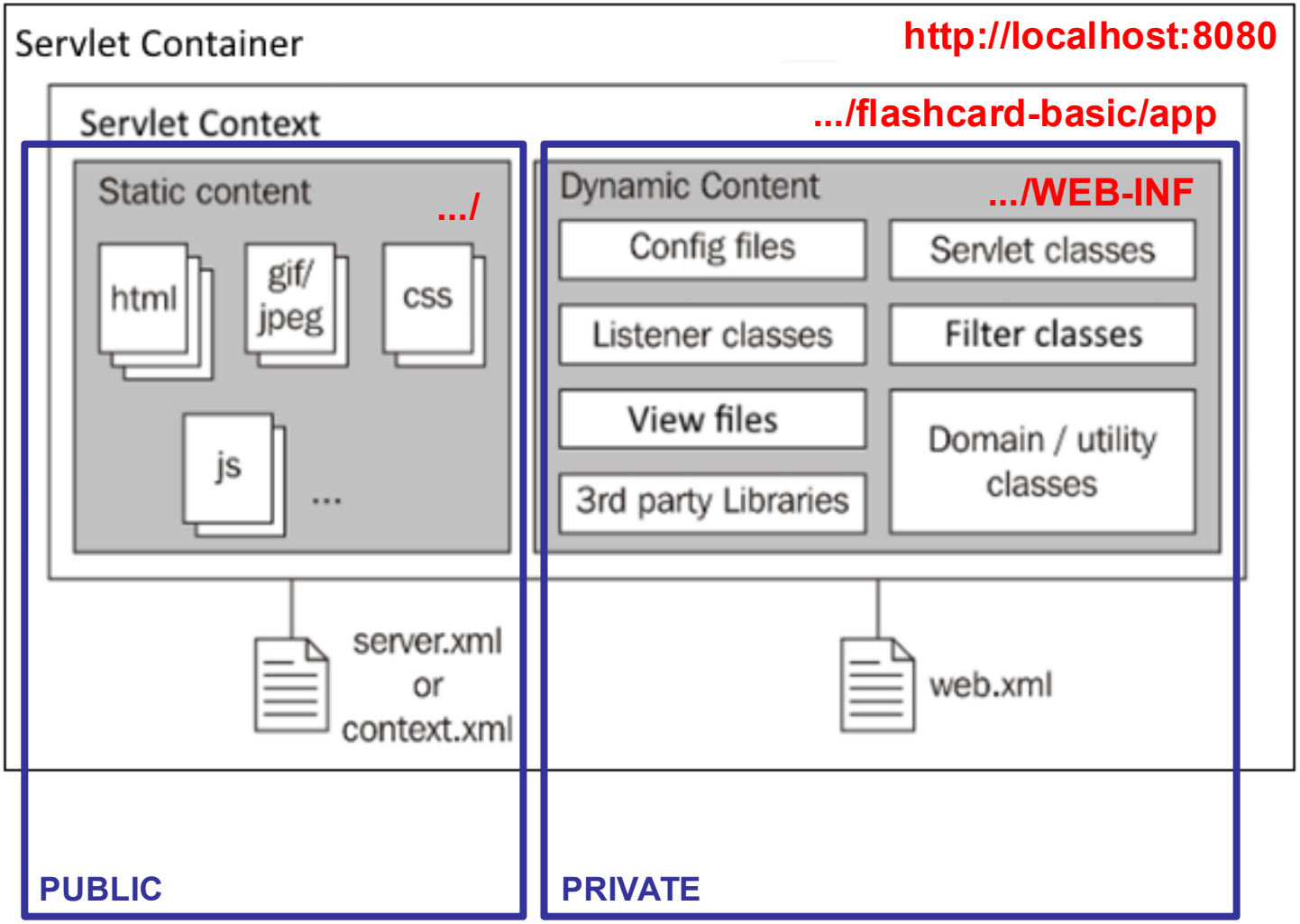
Servlet Context

- Verwaltet Werte, die der gesamten Webapplikation zur Verfügung stehen.

- Zugriff auf die **Parameter** der Webapplikation, die im File "web.xml" initialisiert sind:
  - servletContext.getInitParameter("something")

- Verwaltung von **Attributen** während der Laufzeit:
  - servletContext.setAttribute("myAttr", myValue)
  - servlet.Context.getAttribute("myAttr")

Servlet Container

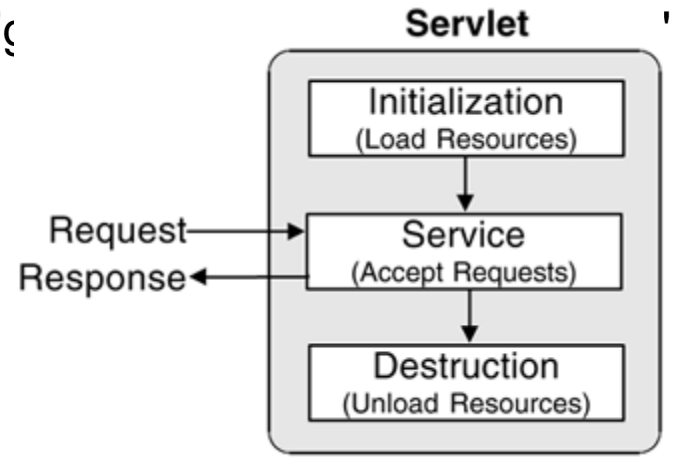


Servlet Container: Zugriff auf Ressourcen

- statische Ressourcen
  - Bilder, Dokumente, ...
  - **über URL direkt zugreifbar (PUBLIC)**
- dynamische Ressourcen
  - die Webapplikation mit den class-Files, den jar-Bibliotheken, ...
  - **über URL direkt NICHT zugreifbar (PRIVATE)**
    → Webapplikation verwaltet die Ressourcen
- Konfiguration der Webapplikation für Zugriff auf statische und dynamische Ressourcen gleichzeitig:
  - URL für dynamische Ressourcen über Servlet Mapping z.B. auf "/app/*" anpassen!

Servlet

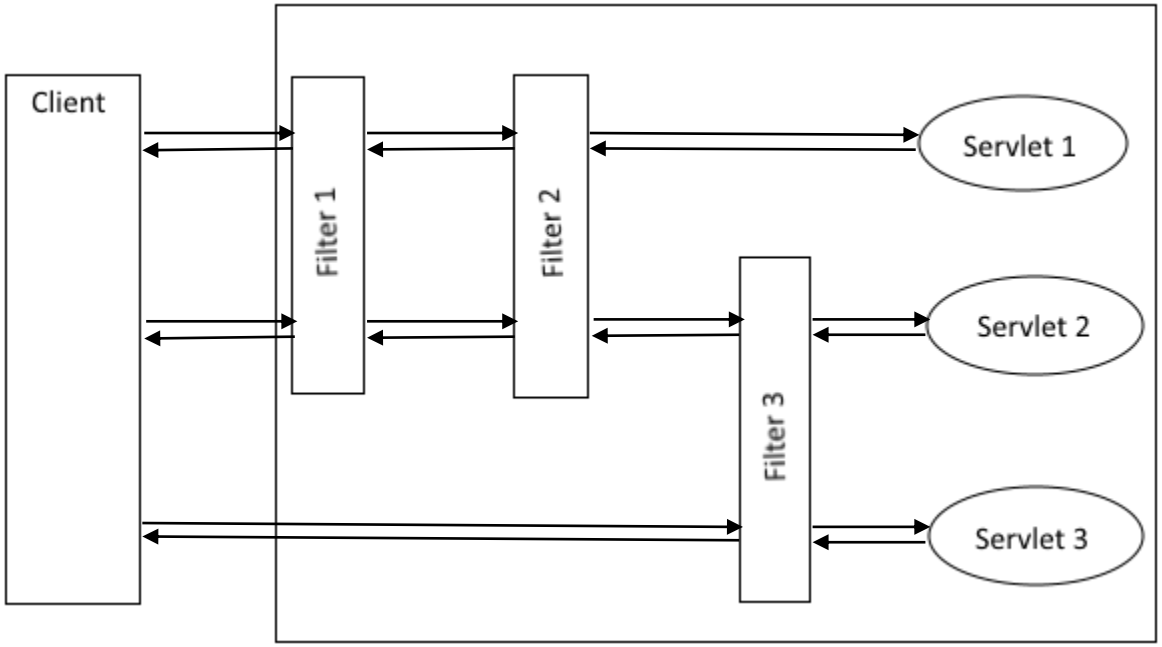
- Das sind die wichtigsten Punkte, die Informatiker/innen von einem Servlet kennen sollten:
  - Zweck eines Servlets
    - Das Servlet setzt einen Use Case um
    - Das Servlet ist der Endpoint eines HTTP-Requests
  - Programmierung eines Servlets
    - Einsatz der Methoden-Parameter "Request" vs. "Response" bei "doXXX()" Methoden. Initialisierung dieser Instanzen immer durch den Servlet-Container.
    - Reihenfolge "ContentType" und Einsatz von "g
  - Laufzeitverhalten eines Servlets
    - Multithreading durch Container
      => "Thread Safety" beachten
      => Instanz-Variablen meiden
    - Konfiguration
      - web.xml vs. Annotation



Komponente "Filter"

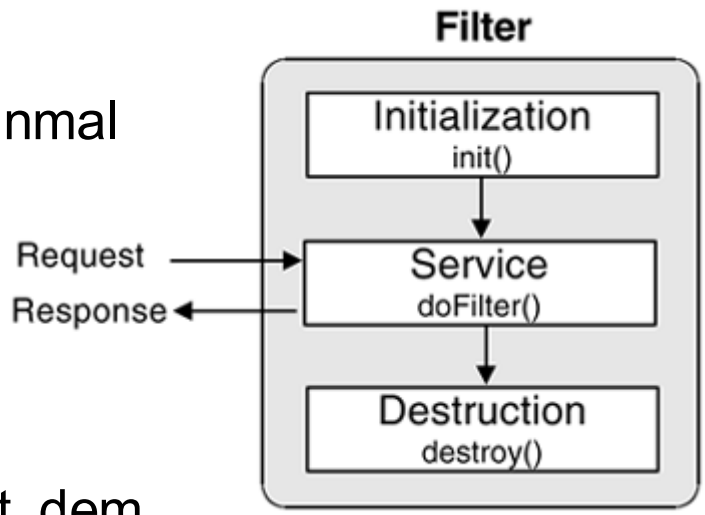
- Zugriff auf Request und/oder Response bei einem Servlet Aufruf, zum Beispiel für
  - Authentication
  - Auditing
  - Image conversion
  - Data compression
- Ausführungsreihenfolge
  - siehe Folie "Filter Mapping"
- Modifikation von Request/Response
  - ResponseWrapper/RequestWrapper (siehe Arbeitsblatt 2.2)

Filter Mapping



- Filter-Reihenfolge bestimmt über
  - Reihenfolge des Mapping-Eintrags im web.xml
  - ... und bei Annotationen?
    - Reihenfolge kann NICHT über Annotation beschrieben werden.
      → **Eintrag "filter-mapping" wird notwendig bleiben**

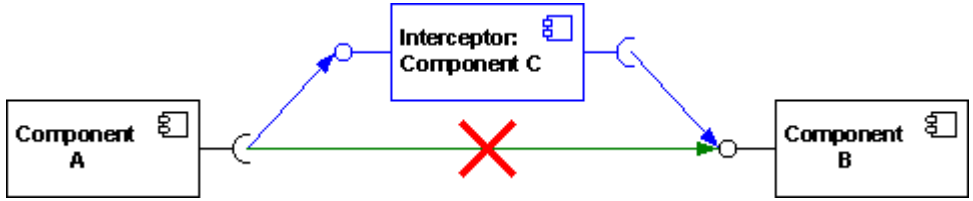
jakarta.servlet.Filter



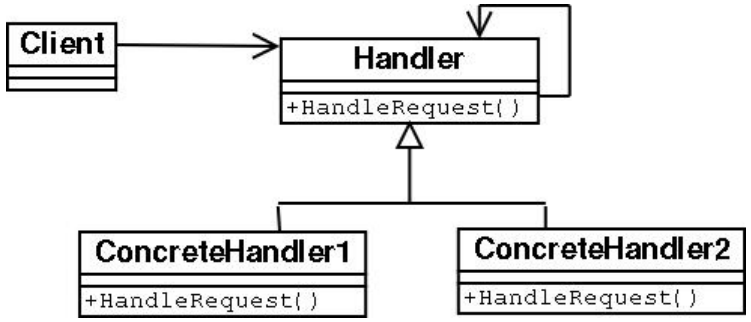
- init(FilterConfig config)
  - Bei der Initialisierung des Filters, wird init() einmal aufgerufen
- **doFilter(ServletRequest request, ServletResponse response, FilterChain filterChain)**
  - Der Servlet-Container ruft bei jedem Request, dem ein Filter zugeordnet ist, die Methode doFilter() auf.
- destroy()
  - Wenn ein Filter aus dem Servlet-Container entfernt wird, wird die Methode destroy() aufgerufen und der Filter kann allfällige Aufräumarbeiten erledigen.

Design Patterns hinter dem Konzept "Filter"

- **Interceptor Pattern**
  - Interceptors sollen technische Dienste bereitstellen, die nicht direkt mit der Anwendungslogik zu tun haben.



- **Chain of Responsibility Pattern**
  - Der Auslöser und der Verarbeiter einer Nachricht werden entkoppelt. Es wird dabei eine Kette von Objekten durchlaufen, welche die Nachricht verarbeiten können. Die Nachricht wird so lange weitergereicht, bis ein Objekt diese verarbeitet hat oder das Ende der Kette erreicht ist.



Arbeitsblatt 2.1 "Filter"

- Lesen sie die Theorie zur Filter-Komponente
- Lösen sie die Aufgabe 1: BasicFilter implementieren
  - Bemerkungen:
    - doFilter(ServletRequest request, ...)
      Der Parameter "request" ist in einer Webapplikation vom Typ "**HttpServletRequest**". Dieser Typ bietet verschiedene Methoden, die im Type "ServletRequest" nicht sichtbar sind
      → Cast auf "HttpServletRequest" ist sinnvoll
    - **NICHT VERGESSEN**
      - chain.doFilter()
      - Filter Mapping

Komponente "Listener"

- Mit einem Listener kann man in der Webapplikation auf verschiedene Ereignisse des Servlet-Containers reagieren, wie:
  - Ereignisse im "Servlet Context"
    → Klasse ServletContextListener
  - Ereignisse bei den "Servlet Context Attributes"
    → Klasse ServletContextAttributeListener
  - Ereignisse bei den "Http Session Attributes"
    → Klasse HttpSessionAttributeListener
  - Ereignisse im "Http Session Binding"
    → Klasse HttpSessionBindingListener
  - und weitere...

Arbeitsblatt 2.1 "Listener"

- Lesen sie die Theorie zur Listener-Komponente
- Lösen sie die Aufgabe 2: BasicListener implementieren
  - Bemerkungen:
    - Verwenden sie die Klasse "ServletContextListener"
    - Über "ServletContextEvent" (Input-Parameter der Methode contextInitialized) haben sie mit der Methode "getServletContext()" Zugriff auf den ServletContext, der auch im Servlet selber sichtbar ist.
    - **NICHT VERGESSEN**
      - Eintrag in web.xml oder Annotation
      - Anpassung in "BasicServlet" vornehmen

Arbeitsblatt 2.2: I18NFilter

- **Ergänzungen**
  - **WICHTIG:**
    Es ist auf **dem Outbound nicht möglich den Original Response direkt zu manipulieren**, da mit dem Abschluss der service()-Methode im Servlet der Servlet-Container implizit ein flush() & close() auf den OutputStream aufruft.
    Somit ist eine Modifikation der Response direkt ausgeschlossen - und deshalb kommt die folgenden Wrapper Klasse ins Spiel:
    - HttpServletResponseWrapper
      Provides a convenient implementation of the HttpServletResponse interface that can be subclassed by developers wishing to adapt the **response from a Servlet**.